
GYSMO

Stéphane Gibout – Cédric Arrabie

Table des Matières

1	Introduction	4
I	Manuel de l'utilisateur	5
II	Manuel du développeur	7
2	Services de base	9
2.1	commands	9
2.2	datastore	9
2.2.1	pull	9
2.2.2	push	9
2.2.3	get_last	10
2.3	mail	10
2.3.1	send	10
2.4	models	10
2.5	mqtt	11
2.5.1	Fonctionnement interne	11
2.6	plugin	11
2.6.1	Caractéristiques d'un plugin	11
2.6.2	Service associé	12
2.7	system	12
2.8	task	12
2.8.1	Caractéristique d'une Task	12
2.9	web	12

	3
3 Conclusion	14
III Exercices	15

1. Introduction

Première partie

Manuel de l'utilisateur

Deuxième partie

Manuel du développeur

2. Services de base

Les services de bases sont implémentés dans le package `core`. Ils sont nécessaire au fonctionnement de l'application et se configurent par l'intermédiaire de variables d'environnement (et/ou du fichier `.env`).

2.1 `commands`

Non implémenté pour le moment.

2.2 `datastore`

Permet l'accès aux données dans une optique de traitements «externes» ie pour lesquels les données sont identifiées par leur nom (et pas leur identifiant unique).

2.2.1 `pull`

retourne le dictionnaire des données actuelles sous la forme `{ nom: valeur}`

2.2.2 `push`

reçoit un dictionnaire de la la forme `{ nom: valeur}` et déclenche les traitements nécessaires.

La première étapes consiste à récupérer le device correspondant au nom puis :

- si le device est une source de donnée (capteur), la valeur est transmise sur le topic `uid/data`, qui déclenchera ensuite la mémorisation de la valeur (cf. plus bas). ^esi le device est un actionneur alors la valeur est mémorisé en base et publié sur le topic `uid/action` pour traitement par le device (actionneur).

L'évolution prévue du code impliquera qu'un device actionneur devra publier son état actuel de façon périodique et se comportera donc nécessairement comme un capteur. Le device devra donc évoluer pour mémoriser la dernières valeurs reçue (sens D->G) et publiée (sens G->D). De même, on devra mémoriser l'historique des deux valeurs.

On modifie la table `DEVICE` pour mémoriser la dernière valeurs `GET` et la dernière valeur `SET`. Dans la table `RECORD`, on ajoute un booléen pour séparer les valeur `SET/GET` (ex : action). Attention car pour le moment, la «valeur» associée à une variable est défaut la valeur lue ou la valeur demandée. Pour un actionneur, il faut ajouter un moyen d'accéder à la valeur réelle et/ou la valeur demandée. Je trouver personnellement plus logique de considérer la "valeur" comme celle retournée par le device. Il faudra peut-être trouver une astuce pour remonter aux valeurs de l'action demandée.

2.2.3 `get_last`

Retourne un tuple comportant le nom, la valeur et la date de la donnée dont le nom est passé en paramètre.

Cette fonction devrait être supprimée et fusionnée avec la méthode `pull` en y ajoutant le paramètre facultatif `Fait`.

2.3 `mail`

Permet l'expédition de mails. La configuration se fait par l'intermédiaire des variables suivantes :

- `MAIL_SERVER`
- `MAIL_PORT`
- `MAIL_USERNAME`
- `MAIL_PASSWORD`
- `MAIL_USE_TLS`
- `MAIL_USE_SSL`
- `MAIL_SENDER`
- `MAIL_SUBJECT`
- `MAIL_RECIPIENTS`

2.3.1 `send`

Permet d'expédier un message. Le message HTML est construite à l'aide du template défini dans `core/mail/templates`.

2.4 `models`

Déclaration de la base de données et de toutes les entités manipulées.

La sélection/configuration du SGBDD est réalisé dans le fichier `core/models/common.py` et dépend (selon le type) des variables d'environnement suivante :

- `DB_FILE` (spécifique à `SQLITE`)
- `DB_NAME`
- `DB_URL`
- `DB_PORT`
- `DB_USER`
- `DB_PASSWORD`

Les modèles/entités sont à décrire ici.

2.5 mqtt

Gère la communication MQTT. Rappelons les différents topics :

- `<uid>/data` : message émis par un device à l'attention de Gysmo. La réception de ce message génère la mémorisation de la valeur, ainsi éventuellement l'exécution d'une tâche.
- `<uid>/action` : message émis par Gysmo à l'attention d'un device afin de modifier son état.
- `<uid>/command` : message émis par Gizmo à l'attention d'un device afin de modifier sa configuration.
- `system/register` message transmis par un device vers Gysmo afin de se déclarer (généralement s'il est inconnu).

Ce dernier message est à réfléchir dans la mesure où la configuration d'un device peut être modifiée dynamiquement. Est-ce que ce topic pourrait servir de validation, en acceptant la réception d'une information partielle ?

2.5.1 Fonctionnement interne

Chaque topic est associé à une fonction qui prend comme arguments le nom du topic et la payload (chaîne).

Le topic est ensuite convertie en regex de façon à appeler la fonction correspondante au bon topic. Cette approche permet d'étendre les traitements MQTT à des cas non encore définis (comme l'interfacage avec TTN par exemple).

Le module MQTT souscrit à minima à deux topics qui sont `+/data` et `system/register`. Le premier déclenche la mémorisation de la valeur transmise si le device est connu et actif. Si le capteur n'est pas connu, un message REGISTER est transmis au device correspondant via le topic `uid/command`. Si le device est connu mais non actif, le message est ignoré. Le second permet de créer un device en base, qui devra être validée par un utilisateur.

2.6 plugin

Ce module gère les plugins (instances d'une sous classe `plugin`).

Un plugin permet d'ajouter des fonctionnalités à GYSMO par l'intermédiaire d'une interface web (webapp) et de tâches (task) déclenchées périodiquement et/ou suite à la modification d'une donnée. Webapp et task sont facultatives.

2.6.1 Caractéristiques d'un plugin

Un plugin est défini par :

- `pid` un identifiant unique
- `prefix` un chemin de base pour l'application web associée
- `menu_name` le texte de l'entrée du menu dans l'interface web principale.

La classe «mère»`Plugin` est abstraite et certaines de ces méthodes doivent être surchargées :

- `start()` qui est lancée au démarrage du système. Ne fait rien par défaut
- `stop()` qui est lancée à l'arrêt du système. Ne fait rien par défaut
- `webapps()` Retourne la liste des webapps définies par le plugin. Retourne une liste vide par défaut.

- `tasks()` Retourne la liste des tasks définies par le plugin. Retourne une liste vide par défaut.

Pourquoi la méthode `sync` n'est pas à ce niveau ?

2.6.2 Service associé

Le service gère les plugins. Il permet en particulier d'enregistrer un plugin à l'aide de la méthode `add`.

Les méthodes `start` et `stop` ne font qu'appeler les méthodes des plugins référencées. La méthode `sync` appelle la méthode correspondante des tasks.

On a ici un beau petit mélange qu'il va falloir corriger, notamment :

- la méthode `sync` doit appartenir à plugin et non à task.
- l'ajout ne doit pouvoir se faire que si le système n'a pas démarré
- Est-ce que plugin ne devrait il pas fusionner avec system ?

Nettoyage effectuée. C'est un peu plus propre, jusqu'à la prochaine idée

2.7 system

Gère les actions de haut niveau (démarrage et arrêt) ainsi que l'ajout de plugin (?) via le fichier de configuration (`plugin.yaml`).

La méthode `add_plugin` n'a rien à faire ici

2.8 task

Le service gère l'exécution des tâches.

2.8.1 Caractéristique d'une Task

Une tâche est définie par son identifiant unique (`id`, `String`) et l'identifiant du plugin auquel elle est rattachée (`pluginid`, `string`).

Le déclenchement peut être forcé (méthode `fire`) mais également périodique (`set_period`) où à la modification d'un paramètre (`add_regex`).

Le traitement à proprement dit est défini en redéfinissant la méthode `execute` (obligatoire). les autres méthodes (du cycle de vie) sont `start`, `stop` et `sync` Pas certain que cela doivent être ici...

2.9 web

Gère le serveur web interne et dont l'exécution des webapps. Jinja est utilisé comme moteur de templating. Les variables globales définies sont :

- `__ts__` : l'instant courant
- `__pid__` : l'identifiant unique du plugin si pertinent
- `__menu__` : l'entrée menu associé au plugin si pertinent.

Deux méthodes ont également été ajoutée :

- `dynamic_url_for` : retourne l'url relative à la webapp du plugin
- `dynamic_url_or_hash_for` : idem si dessus mais retourne un `#` si la route n'existe pas.

Ces deux méthode de template sont nécessaire car le point de montage des webapp (blueprint) n'est connu qu'après le démarrage du serveur web. Elles sont également utilisables dans le code.

Par commodité, l'identifiant du plugin (plugin) est ajouté à l'objet request.

3. Conclusion

Troisième partie

Exercices

